

From a specification to running tests

A walkthrough you can do at home

Two worked examples: IHE IUA, then IHE PCF

The IUA numbers in this deck were measured. The PCF half is the exercise — slide 20 says exactly which is which.

Where this ends up

A FHIR implementation guide carrying TestPlan resources. Not a folder of scripts — a publishable package that names its tests as FHIR:

```
ihe-iaa-test/  
  fsh-generated/resources/  
    TestPlan-iaa-authorization-server-tests.json    13 tests  
    TestPlan-iaa-resource-server-tests.json         10  
    TestPlan-iaa-authorization-client-tests.json    5  
    Binary-*-gherkin-script.json                   the scripts they name  
  dist/package.tgz      ← installable: ihe.iti.iaa.test
```

```
28 tests, 94 assertions, 7 resources      SUSHI: 0 errors
```

Getting there has four steps, and the first three need nothing unusual:

scope → author → compile → publish as an IG

What you need: Node, and two npm packages. **Not needed:** Docker, a running test bed, or membership of any organisation.

The cast, and who does what

	What it is	What it does here
You	—	Decide scope. Confirm the case list. Sign off.
Agent	Claude Code	Reads the spec, writes Gherkin, runs the compiler, fixes what it broke
Skills	<code>.claude/skills/*</code>	The procedure the agent follows: stages, gates, traps
CLI + library	<code>@opentestbed/otb-gherkin</code>	Compiles Gherkin → GITB TDL. The only hard dependency
Dialects	<code>components/<id>/</code>	Verbs for one domain: OAuth, JWT, FHIR validation
Workbench	the authoring app	Optional. Browser editor for the same files
Test Bed	ITB + Docker	Optional, and only needed to actually <i>run</i> tests

The last two being optional is the point. Writing tests and running tests are separate jobs with separate prerequisites.

Setup

```
mkdir iua-test-suite && cd iua-test-suite  
npm init -y  
npm install @opentestbed/otb-gherkin @opentestbed/dialect-fhir-validator
```

Then get the skills — they are what makes the agent follow a procedure instead of improvising:

```
git clone https://github.com/OpenTestBed/itb-plugin-authoring  
cp -r itb-plugin-authoring/.claude/skills .claude/skills
```

That is the whole setup. No containers, no services, nothing to start.

Journey 1 — IHE IUA

Internet User Authorization

The hard case: it teaches you what the language is missing

Prompt 1 — hand it the specification

I want to create a test plan for <https://profiles.ihe.net/ITI/IUA/index.html> and all its volumes

What happens between the agent and the spec

The agent invokes the `spec-to-tests` skill, which imposes four stages with gates: *scope* → *agree* → *author* → *prove*. It will not let itself skip to writing Gherkin.

Then it reads the actual specification and finds:

- a single page carrying **both volumes**, narrative only;
- **no FHIR package** — `ihe.iti.iua` is not on the registry;
- **so no profiles, no capability statements, no examples.**

Every requirement has to be read out of prose. That decides the shape of everything that follows.

What it comes back with — the scope

Three actors, four transactions, four options:

Actor	Transactions
Authorization Client	ITI-71 Get Access Token, ITI-72 Incorporate Access Token
Authorization Server	ITI-71, ITI-102 Introspect, ITI-103 Metadata
Resource Server	ITI-72, ITI-102, ITI-103

Your decision, not the agent's: actor-scoped (one plan per actor) or profile-scoped (does an instance conform)? IUA has no profiles, so actor-scoped is the only honest answer here.

The agent writes the scope down — including **what is deliberately out** — before writing a single test. Here: the SAML Token Option, signature verification, and transport security, which belongs to ATNA.

Prompt 2 — the case list, before any code

Positive paths and the negative must-reject cases. Operator attestation where the profile requires something displayed.

The agent produces **28 one-line cases**, grouped, each naming the requirement it covers:

```
RS-02  An expired token is refused with 401
      ITI-72: "shall validate or introspect the access token
            and ensure that it has not expired"
AS-06  An unknown grant_type is refused with unsupported_grant_type
      OAuth 2.1 §5.3
```

A case that cannot name a requirement gets **dropped**, not kept. And a list is cheap to change — a suite is not. This gate exists so you argue about coverage now rather than after 28 scenarios exist.

You confirm the list. Then it writes.

Then it hits a wall — and this is the interesting part

The agent writes the Authorization Server plan. It **compiles clean**.

And it is wrong.

```
Given set header "Content-Type" to "application/x-www-form-urlencoded"
When Client posts to AuthorizationServer at "/token" with:
    """
    grant_type=client_credentials
    """
```

The core language's POST verbs assign `Content-Type: application/json` *inside* the verb — so the header set a line earlier is **silently overwritten**. Every token request goes out as form-encoded content labelled JSON.

A clean compile means the sentences were understood. Not that the requests are correct. Keep this slide in mind whenever you see `0 errors`.

Two things the core language genuinely cannot do

1. **Send** `application/x-www-form-urlencoded` — the POST verbs fix the content type internally.
2. **Build a body from run-time values.** A token exchange needs `grant_type=authorization_code&code=` + *the code the operator just pasted*.
The core's `{value}` slot takes a literal, a `$variable` or a number — never a `concat()`.

Both are deliberate: the core is shaped for JSON APIs.

Neither is something a test author can work around.

So this is not a reason to write worse tests. It is a dialect.

Prompt 3 — teach the language to speak OAuth

The core can't send form-encoded bodies or build one from a variable. Add an oauth dialect with the verbs these tests need, and keep it local — I'm not publishing anything.

The agent invokes **add-language-dialect** and writes two YAML files:

```
components/oauth/  
  component.yml    identity, which core language it targets  
  steps.yml        the verbs
```

No repository. No npm package. No organisation membership. The compiler cannot tell the difference between a dialect from npm and one you typed five minutes ago.

What a verb actually looks like

```
- text: '{actor} requests a token from {actor} at {string} with grant {value}( and scope {value})?'
  actions:
    - assign: { to: 'jsonHeaders{Content-Type}', value: '"application/x-www-form-urlencoded"' }
    - assign: { to: 'oauthBody', value: 'concat("grant_type=", $4)' }
    - assign: { to: 'oauthBody', value: 'concat($oauthBody, "&scope=", $5)' }
      when: '$5.raw'
    - send: { handler: 'HttpMessagingV2', to: '$2', inputs: { body: '$oauthBody', ... } }
```

Two details that matter:

- it writes into the core's **own** header map, so `set header` and `set bearer token from` keep working beside it;
- `when: '$5.raw'` adds `&scope=` **only** if the author wrote a scope — so an omitted parameter is absent, not empty. A PKCE negative test depends on exactly that.

And the test now reads like the requirement

```
Scenario: AS-09 a code exchanged without its verifier is refused as invalid_grant
  Given Client is asked for $freshCode with "Paste a fresh authorization code"
  When Client exchanges the code $freshCode for a token on AuthorizationServer at "/token"
  Then $response.status should be 400
  And $response.body should be the OAuth error "invalid_grant"
```

No `code_verifier` is sent at all, because the slot was left off. That is the test.

A second dialect, `jwt`, decodes the issued token so nine cases can assert on its claims — `iss`, `sub`, `aud`, `jti`, `exp`, `iat`, `scope`.

Prompt 4 — prove it

Compile all three feature files and show me the generated TDL for the form bodies and the bearer token.

```
0 iua-authorization-client.feature [5 test cases]
0 iua-authorization-server.feature [13 test cases]
0 iua-resource-server.feature      [10 test cases]
```

```
clean: 3, with errors: 0
```

Why it asks to see the TDL — compiling is not enough

```
<assign to="jsonHeaders{Authorization}">$clientAuth</assign>
<assign to="jsonHeaders{Content-Type}">"application/x-www-form-urlencoded"</assign>
<assign to="oauthBody">concat("grant_type=", "client_credentials")</assign>
```

Client auth survives; the content type is right; the body carries only the parameters that scenario wrote. **That** is the check — not 0 errors .

Journey 2 — IHE PCF

Privacy Consent on FHIR

The easy case — and it is easy for a reason worth understanding

Prompt 1 — the same opening prompt

Now do the same for <https://profiles.ihe.net/ITI/PCF/> — I want a test plan for all its volumes.

The agent runs the same skill, the same four stages. What it finds is **completely different:**

	IUA	PCF 1.1.0
FHIR package	none	<code>ihe.iti.pcf</code> , FHIR 4.0.1
Capability statements	none	3, one per actor
Profiles	none	3 Consent profiles
Requirements from	prose	prose + machine-readable artifacts

What PCF hands the agent for free

Three actors, each with a capability statement that **states its obligations**:

Actor	<code>Consent</code> interactions
Consent Recorder	create, read, update, delete, search-type
Consent Registry	create, read, update, delete, search-type
Consent Authorization Server	read, search-type

Three profiles of increasing strictness: `consentBasic` ,
`consentIntermediate` , `consentAdvanced` .

On IUA the agent had to *argue* for every case from a sentence of prose. Here "the Registry must support `search-type` on `Consent` " is a **fact read from a file**. The case list is derived, not invented.

And the vocabulary already exists

PCF is a FHIR profile specification, so the central assertion is conformance — which the published `fhir-validator` dialect already provides:

```
Scenario: a recorded consent conforms to the Basic profile
  When ConsentRecorder gets from ConsentRegistry at "/Consent/" with id $consentId as $consent
  Then $response.status should be 200
  And $consent should conform to "https://profiles.ihe.net/ITI/PCF/StructureDefinition/IHE.PCF.consentBasic"
```

No new dialect. None.

And the **profile-scoped** shape becomes available — "does this instance conform?" — which on IUA was impossible, because IUA has no profiles to conform to.

The payoff: PCF reuses IUA's work

PCF's volume 1 mentions IUA **116 times** — it builds on it. Consent decisions ride on an access token.

So the moment PCF needs to assert something about *the token carrying the consent decision*, the dialects from journey 1 are already sitting in

`components/` :

```
components/  
  fhir-validator/    from npm – profile conformance  
  oauth/            from journey 1 – token requests  
  jwt/              from journey 1 – claims inside the token
```

This is the compounding effect. The first specification pays for the vocabulary. The second one spends it. The third is nearly free.

Mixing sources — one command, three dialects

```
npx otb-gherkin dialects --installed --out .
```

```
ok   fhir-validator      3 files  @opentestbed/dialect-fhir-validator@2.0.0
kept jwt                  already in components, not from this source
kept oauth                already in components, not from this source

1 dialect in components (+2 kept)
```

Dialect	Where from	Pinned by
fhir-validator	npm	your lockfile
oauth , jwt	your own folders	your git history

kept matters: a dialect the command did not fetch but found in place is **preserved**. Your hand-written work is not deleted by a refresh.

Needs otb-gherkin ≥ 0.4.1. Earlier versions silently unlisted them.

The slide that matters most

Compiling is gate 1. Running is gate 2. They are not the same claim.

✓	Gate 1	– 28 cases compile, 0 errors	MET
✗	Gate 2	– the suite runs against a system	NOT MET

The IUA suite in this deck has **never been executed**. There is no IUA implementation configured to measure. So:

Nothing here is evidence that any system conforms. Nothing here is even evidence that the tests *pass*.

That belongs in the record, in writing, not left to be inferred. A suite that does not say what it leaves out reads as a claim of completeness it cannot support.

And when you do run it

A must-reject case that passes first time deserves suspicion. Feed it something **valid** and confirm it then fails. *A test that cannot fail is not a test.*

Where people need to stay in the loop

Four IUA cases rest on a human, each for a stated reason:

Case	Why a machine cannot settle it
AS-02, AS-08, AS-09	A user must authenticate and consent at a browser
AS-04	<code>exp</code> is epoch seconds; TDL has no clock to compare against
RS-09	The profile never says which claims bind to which transaction
RS-10	Introspection happens server-to-server; invisible to the client

The Resource Server plan also needs tokens it cannot mint — expired, wrong audience, too narrow a scope — so an operator pastes them in once per run.

Writing this down is the deliverable. An automated suite that quietly skips what it cannot check is worse than one that asks.

Traps worth knowing before you start

kinds: is a list, not a mapping. As a mapping it is valid YAML, loads, and used to kill the compiler with `object is not iterable` — no file, no line.

Actor kinds are for infrastructure, never the system under test.

`is the system under test` has no kind slot, so `{actor:my-kind}` can never bind to it. Use a plain `{actor}` .

components/index.json is not decoration. A folder missing from that list is invisible, and the only symptom is `No mapping for step` — blaming your feature file for a problem in your index.

Dialects cannot add sentence shapes, only verbs. That restriction is why every dialect reads alike.

Do it at home

```
# 1. setup
mkdir my-tests && cd my-tests && npm init -y
npm install @opentestbed/otb-gherkin @opentestbed/dialect-fhir-validator
git clone https://github.com/OpenTestBed/itb-plugin-authoring
cp -r itb-plugin-authoring/.claude/skills .claude/skills

# 2. ask the agent
claude
```

I want to create a test plan for <https://profiles.ihe.net/ITI/PCF/> and all its volumes. Use the spec-to-tests skill. Keep any dialect local — I'm not publishing anything.

```
# 3. the loop you will run many times
npx otb-gherkin dialects --installed --out .
ITB_ASSET_ROOT=. npx otb-gherkin compile features
```

Now publish it as a FHIR IG with a TestPlan per actor.

The end result

A FHIR IG with the test plans in it

Prompt 5 — make it a FHIR IG

Now publish this as a FHIR IG with the test plans — a TestPlan per actor, carrying the Gherkin.

The agent invokes `spec-to-test-ig`. Feature files are the working form; a **TestPlan** is the publishable one. The mapping is mechanical:

Gherkin	FHIR TestPlan
one feature file (one actor)	one <code>TestPlan</code> , with its <code>scope</code>
<code>Scenario:</code>	a <code>suite.test</code> , <code>operation: gherkin/Scenario</code>
the <code># ITI-71 ...</code> comment above it	that test's <code>description</code> — the traceability
each <code>Then</code> / <code>And</code> step	an <code>assertion.human</code>
the whole <code>.feature</code> file	a <code>Binary</code> , <code>text/x-gherkin</code>

One command, and what comes out

```
node app/scripts/build-test-ig.mjs app/public/data/iua-test-ig.json --package
```

```
wrote ImplementationGuides/ihe-iua-test:
  sushi-config.yaml, 3 TestPlan(s) with 28 test case(s), 3 feature(s), pages
== sushi ==                                0 Errors
== package ==  dist\package.tgz (14.7 KB)   7 resources, 3 feature files
```

```
{ "name": "AS-01 a client-credentials grant returns a bearer token and its scope",
  "description": "AS-01 – ITI-71. The required response fields: token_type,
                  access_token and scope. expires_in and refresh_token are
                  optional and not asserted.",
  "operation": "gherkin/Scenario",
  "assertion": [ { "severity": "error", "human": "$response.status should be 200" },
                  { "severity": "error", "human": "$tokenType should be \"Bearer\"" } ] }
```

Why a TestPlan rather than just the scripts

The feature file is what runs. The TestPlan is what you can **publish, cite and review**:

- it is a **FHIR resource**, so it installs, resolves and renders like any other IG artifact — `ihe.itl.iua.test` ;
- `scope` names the actor under test, so a reader sees what each plan measures;
- every test's `description` carries **the quoted requirement**, so a reviewer can check coverage without reading Gherkin;
- the `Binary` keeps the executable script attached to the declaration, so the two cannot drift apart.

IUA publishes no FHIR package of its own — `ihe.itl.iua` is not on the registry. So the config sets `spec.dependency: false` and the plans quote the supplement in prose. **A spec does not have to be FHIR for its tests to be.**

The IG the publisher builds

Alongside the resources, the builder writes a real IG project — pages, menu, CI workflow, the IG Publisher scaffold:

<code>input/fsh/testplan-iaa-authorization-server.fsh</code>	the TestPlan, as FSH
<code>input/testing/gherkin/*.feature</code>	the scripts
<code>input/pagecontent/index.md</code>	what this IG is
<code>input/pagecontent/process.md</code>	← IUA-TESTS.md becomes "Method"
<code>.github/workflows/fhirbuild.yml</code>	builds it on push
<code>dist/package.tgz</code>	installable

The record you kept at stage 1 — the scope, the 28 cases, **what is deliberately not covered** — becomes the IG's **Method** page. It was never a side note; it is a published page of the deliverable.

And only now, if you want to run them

```
ITB_ASSET_ROOT=. npx otb-gherkin compile features --out build --zip suite.zip
```

Deploy `build/suite.zip` to an Interoperability Test Bed and run it.

This is where Docker and a running ITB finally appear — and where you need a real implementation to measure. Everything before this slide needed neither.

Note what the IG does and does not claim. It publishes the tests and their traceability. It is not a test *report*: no `TestReport`, no verdicts. Those only exist after a run against something real.

What is measured here, and what is not

Measured, on 1 October 2026: the IUA suite — 28 cases, 3 feature files, 2 dialects written locally, compiling with 0 errors against the published `@opentestbed/otb-gherkin` 0.4.1 from a clean `npm install`; the generated TDL read by hand; and the IG built — 3 TestPlans, 28 tests, **94 assertions**, SUSHI 0 errors, `dist/package.tgz` at 14.7 KB. The suite has **never been run**.

Not measured: the PCF journey. Its *facts* are real and taken from the published IG — package `ihe.iti.pcf` 1.1.0, FHIR 4.0.1, three capability statements, three Consent profiles, 116 references to IUA. The scenarios shown are illustrative. **PCF is your exercise, not a result.**

Told this way on purpose. A walkthrough that presents an unrun suite as a passing one teaches the single worst habit in conformance testing.

Links

Front door, skills, <code>EXTENDING.md</code>	<code>github.com/OpenTestBed/itb-plugin-authoring</code>
Compiler + CLI	<code>npm i @opentestbed/otb-gherkin</code>
OAuth 2.1 dialect	<code>github.com/OpenTestBed/itb-plugin-oauth</code>
JWT dialect	<code>github.com/OpenTestBed/itb-plugin-jwt</code>
IHE IUA	<code>profiles.ihe.net/ITI/IUA/</code>
IHE PCF	<code>profiles.ihe.net/ITI/PCF/</code>

The four ideas worth keeping:

- 1. A clean compile means your sentences parsed — nothing more.
- 2. A missing verb is a dialect, not an excuse for a worse test.
- 3. Write down what you did not cover. That part is the deliverable.
- 4. The end result is a **FHIR IG with TestPlans** — publishable, citable, and traceable to the requirement. A test nobody can cite is a test nobody adopts.